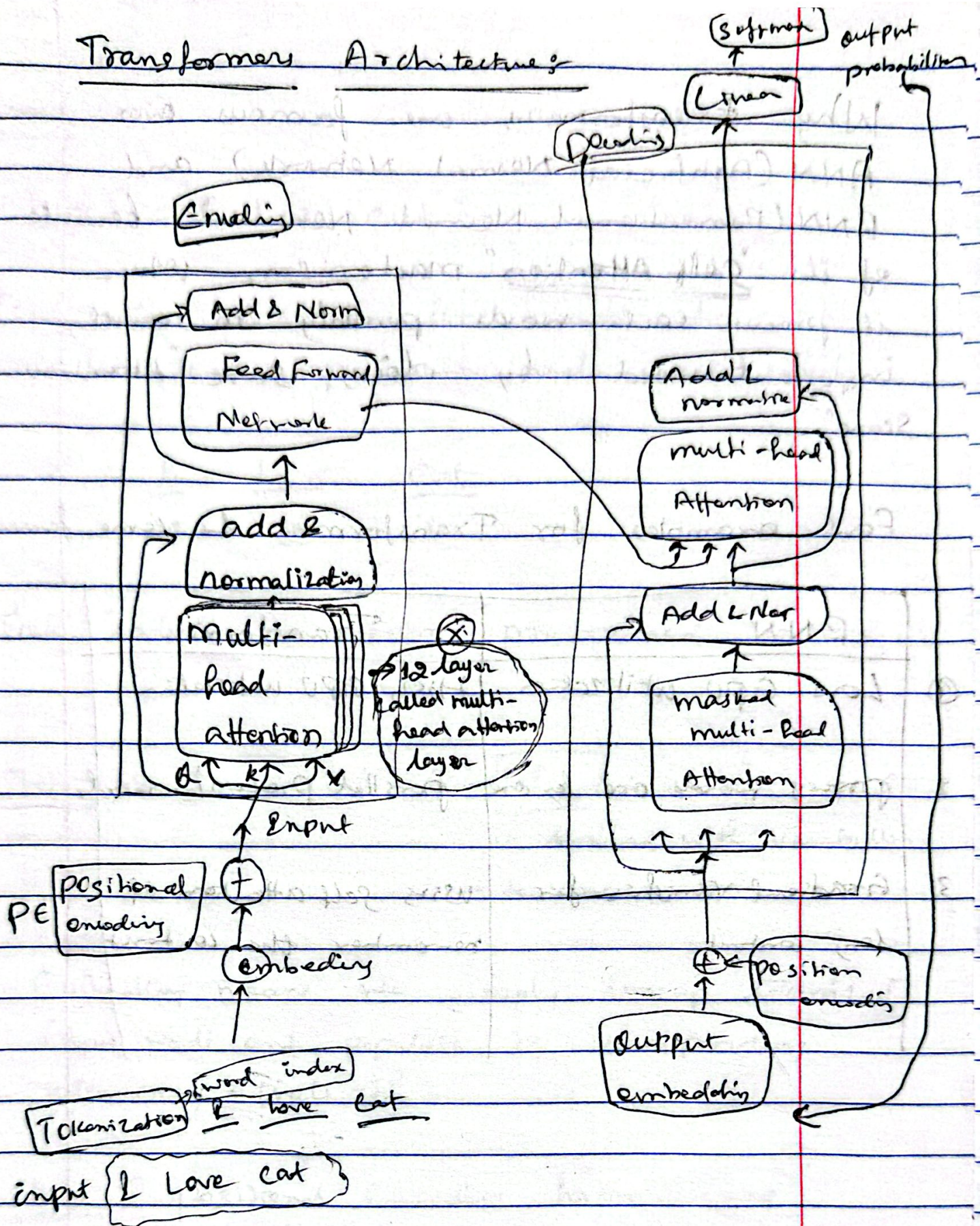


Transformer Architecture



Why transformers are famous over ANN (Artificial Neural Network) and RNN (Recurrent Neural Network) because of its 'Self Attention' mechanism. Where it process each words parallelly to see the important word by doing 'Score' 'Attention Score'.

Few examples for Transformer to come famous

	RNN	Transformer
①	Low GPU utilization	High GPU utilization
2	process words one by one	parallel processing words
3	Gradient vanishes for long content	Using self attention it remember the content

Step 1:-

X Input = "I Love cat"

Tokenization is the first step to split words for further processing and assign index

X = I Love cat
 20 30 40

These index are from pre-trained Vectors

Step 2:- Embedding (W_o) is a vector and it trained with Vocabulary

Embedding means we simply lookup tokenized input words and populate its corresponding vector array that's it.

Step 3:- Positional Encoding formula (Sine)

It injects input ordering into the input starts from 0, 1, 2, 3

its a precalculated unique vector embedding based on its position in the sentence.

positional encoding

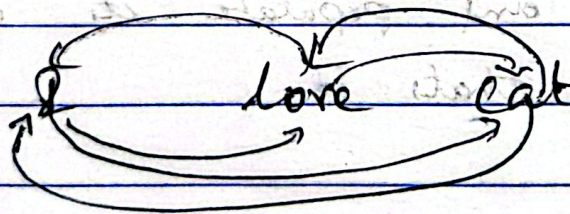
$$X = \begin{bmatrix} 1 \\ 0.1 \\ 5.0 \\ 2.0 \\ 0.1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.1 \\ 0.5 \\ -0.1 \\ -0.8 \end{bmatrix} \quad \text{Love} \quad \begin{bmatrix} 1 \\ 0.2 \\ 0.6 \\ 0.9 \\ -0.1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.1 \\ 0.8 \\ 0.2 \\ 0.4 \end{bmatrix}$$

Once we get the sum of each vector is the input for the Self-Attention heart of Transformers.

Step 4 a:- Attention Layer (multi-head):-

To find out which word has more attention each word influence on the other word to find most influence word by calculating "attention score". These influence happens in "parallelly".

example:-



To calculate the attention score we have to do the following

we need to perform Q, K, V

W_Q - we got this array from training itself.

W_K - got this vector from training.

W_V - got from training itself.

Since we have 12 multi-head layers we have pre trained vectors during training process.

like $W_{Q1}, W_{Q2}, W_{Q3}, \dots, W_{Q12}$

$W_{K1}, W_{K2}, W_{K3}, \dots, W_{K12}$

$W_{V1}, W_{V2}, W_{V3}, \dots, W_{V12}$

Each input array will multiply by each Q, K, V

$$\begin{matrix} x_1 & x_2 & x_3 \\ \text{Love} & \text{cat} & \text{cat} \end{matrix}$$

$$\begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \begin{bmatrix} 0.2 \\ 0.5 \\ 0.2 \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.2 \\ 0.0 \end{bmatrix}$$

x_1

$$\begin{matrix} x_1 & W_{Q1} & & W_{K1} & & W_{V1} \\ \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} & \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} & & \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} & \times \begin{bmatrix} 0.2 \\ 0.4 \\ 0.5 \end{bmatrix} & & \begin{bmatrix} 0.1 \\ 0.2 \\ 0.5 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \end{matrix}$$

x_2 Love

$$x_2 \quad W_{k1} \quad x_2 \quad W_{k1} \quad x_2 \quad W_{k1}$$
$$\begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \quad \begin{bmatrix} 0.2 \\ 0.3 \\ 0.4 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \quad \begin{bmatrix} 0.4 \\ 0.1 \\ 0.0 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$$

x_3 eat

$$x_3 \quad W_{k1} \quad x_3 \quad W_{k1} \quad x_3 \quad W_{k1}$$
$$\begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \quad \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \quad \begin{bmatrix} 0.1 \\ 0.3 \\ 0.2 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$$

There above x_1, x_2, x_3 with w_{k1}, w_{k2}, w_{k3} for "head1" Similarly same calculation happens for different head2 ($x_1 \times w_{k2}, x_2 \times w_{k2}, x_3 \times w_{k2}$) head3 ($x_1 \times w_{k3}, x_2 \times w_{k3}, x_3 \times w_{k3}$) :--- head12 ($x_1 \times w_{k12}, x_2 \times w_{k12}, x_3 \times w_{k12}$) :--- parallelly same calculation happens with different ($w_{k2}, w_{k3}, w_{k12}, w_{k13}, w_{k14}, w_{k15}$) pre-trained vectors.

To calculate the attention score we need to multiply

$Q \times K$ This a "dot product" between Q and K .

we do multiplication with each row of Q to each row of K values.

Finally we will know how one word is relevant to each other.

The higher number is the "Attention" word.

Step 4b:- Once the multiplication is done we will have final vector where each vector know each other and their relevance. the most relevant "numbers will be higher" and less important will be lower while doing so the least will be ignored to fix the we have to do "Scale" fix to keep the number manageable

$$I = \left[\frac{0.51}{1.4}, \frac{0.5}{1.4}, \frac{0.21}{1.4}, \frac{0.43}{1.4} \right]$$

$$d_k = 2$$

$$d_v = 1.4$$

$$\text{Love} = \left[\frac{0.4}{1.4}, \frac{0.7}{1.4}, \frac{0.8}{1.4}, \frac{0.2}{1.4} \right]$$

$$\text{Cat} = \left[\frac{0.9}{1.4}, \frac{0.2}{1.4}, \frac{0.4}{1.4}, \frac{0.3}{1.4} \right]$$

By doing this we normalize all the word and their relevance it will not change the "Attention word" but it will not lose the low attention words

Step 4c:- Soft max:

Soft max:-

$$\text{ex: } \frac{e^{x_1} \rightarrow e^{x_2} \rightarrow e^{x_3}}$$

Now by this soft max $e = 2.78$
formula we calculate percentage Euler's number constant
how much percentage each word is attention.
adding all the sum should come to "1".

Finally,

W_0 - pre-trained vector from training

I gets 46% of the attention

how gets 30% of the attention

Cats gets 24% of the attention

→ same this happens for all other 12 words and letters

⊗ concat(head 1, head 2, head 3, ..., head n) $\times W_0$

(mean avg of value)

Step 4d:- Add & Normalization

norm(x) =

$$\frac{x - \mu}{\sigma}$$

First we add original input for the transformers + output of the attention (x) output

This help to keep track of the input along with attention output.

Now apply Layer Normalization

This normalization does not go through normalization or too small.

Step 5: Feed Forward Network (FNN) is a neural network.

Input (4)



Linear layer (16) to think to get large probabilities



Relu (kill negative values)



Linear layer (4) shrink back



output

	Low	cat
0.1	0.5	0.5
0.2	0.7	0.7
0.3	0.9	0.9
0.4	0.9	0.9

This output is the most refined output from the transformer.

Step 5a: Add + Normalization

FNN input + FNN predicted output we do sum for these two.

then Normalise to control output values

Now, we have final output from
encoder "25th contextual vector"

Let's move to decoder now.

To predict the next word we care about the
last word context only. This Decoder predicts the
next word.

$$\begin{bmatrix} 0.1 \\ 0.2 \\ 0.4 \\ 0.5 \end{bmatrix}$$

Decoder Translation:-

Decoder translation generate word by word only.

Step 1: [<Start>]

↓

Tokenize using FRENCH Vocabulary

↓

Embed using the FRENCH + positional

Embedding

↓

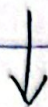
masked self-Attention → only see previous outputs

↓

Cross-Attention

Q - from decoder - what should I translate next

K, V from encoder



Feed Forward Network



Linear → Softmax over FRENCH Vocabulary

→ predict French word



predicted "J'aime" →

Repeat from step 1 along with "J'aime"

Step 1: [Start], J'aime



continue same process

Final: "J'aime les chats"

Next word prediction

⊗ NO one encoder only Decoder.

Step 1: process all the entered sequence input



Tokenizer using one vocabulary



Embedding + positional encoding



masked - self attention

⌈

⌈, here

⌈, here, here

← "only see previous word"



no-ones Attention

X

doesn't exist



FFN



Take the last word ("at") value



Linear → Softmax over same vocabulary



predicted word,

Save step again

input:

"I love cats very"



Embed + positional encoding



masked Self Attention



FFN



Take last word "very"



predict: much



same process ---

:

predicted: . (END) → stop!